

U: Law-Bearing Operations

Explicit Semantic Composition and Evidence-Bound Computation

Edward Ellis

September 2026

Research preprint

Abstract

U is a research language and semantic substrate organized around operations whose composition laws, resource interfaces, observations and evidential obligations remain explicit. A six-constructor structural calculus connects admitted generators without identifying their mathematical models. Seven theory families account for constructive values and proofs, resources and memory, relations and search, measures and probability, behaviors and clocks, continuous dynamics, and coherent state with observation instruments. Q's current mathematical profile uses complex Hilbert spaces and quantum channels; its architectural role is to preserve joint-state composition and observation contracts. The proposal separates an operation's meaning from its realization and from the evidence produced by that realization. It requires checked composite profiles or adapters at mixed-theory boundaries, deterministic elaboration, conservative independence witnesses, distinct artifact identities, and residual-preserving source ingestion. We give conditional structural-preservation results, a finite resource nonduplication argument, a relational adapter-composition theorem with explicit assumptions, a quantitative error-composition bound, identity and invalidation lemmas, and an earned-artifact non-fabrication theorem. The Coherence-Delta Calculus supplies a demanding case study: its existing finite maps, scheduled resets and nested updates must be preserved before their derivatives can authorize recurrence-gated return analysis. CDC predates U; the specialization is a new implementation relationship. The compiler, structural graph, standard libraries, proof kernel and CDC analysis are implemented in U and compile to native functions. The compiler reproduces its translation through three executable stages without a Python runtime. The mathematical results concern stated abstract models; self-compilation and executable conformance checks provide separate implementation evidence. Complete foreign-language subsumption, general dependent type soundness and physical-backend execution remain outside the established scope.

Keywords: programming-language semantics; typed operations; semantic preservation; resources; proof checking; hybrid systems; provenance.

Repository: <https://github.com/ETellis/U>. Companion system: <https://github.com/ETellis/BiDi>.
Interactive introduction: <https://etellis.github.io/U-web/>.

Contents

1 Problem and contribution	3
1.1 Three discriminating examples	3
2 Semantic objects and admission	4
2.1 Admission is a product of dimensions	4
2.2 Configuration and outcomes	4
3 Structural calculus and deterministic elaboration	5
3.1 Local obligations imply a structural theorem	5
3.2 Whole-surface elaboration	6
4 Resources and conservative independence	6
5 Adapters, assumptions and approximation	7
6 A total proof boundary	8
7 Identity, dependencies and invalidation	9
8 Earned artifacts and guarded consequences	10
8.1 Effects and durability	10
9 Residuals and native semantic possession	11
9.1 A fragment-indexed possession criterion	11
10 CDC specialization and compatibility	12
10.1 Synchronous flow and its actual Jacobian	12
10.2 Commit and nest in a fixed discrete itinerary	12
10.3 Scheduled events versus localized guards	13
10.4 Tangent, recurrence and return	13
10.5 Framework, toolchain and RFTC boundaries	14
11 Native implementation and verification	14
11.1 Source, graph and native execution	14
11.2 Executable library scope	14
11.3 Reproducible self-compilation	15
11.4 Discriminating checks	16
11.5 Reproduction and provenance	16
12 Related work and research position	16
13 Limitations and further work	17
A Theory accounting and refusal cases	18
B Formal claims and their exact boundaries	19
C Source and reproducibility record	19

1 Problem and contribution

A common representation is useful only if it preserves the distinctions that make the represented systems meaningful. An integer value, a mutable allocation, a search relation, a probability measure and a quantum channel can all appear behind a function-shaped interface. That fact does not authorize the same rules of copying, observation, reordering or execution for all of them. Similarly, a numerical trajectory and a differential equation may be connected without denoting the same mathematical object.

U takes the operation at an interface as its structural unit. An operation includes a semantic profile, the resources and effects visible at its boundary, the observations under which its behavior is assessed, and the rules by which it may compose with other operations. A realization implements that contract in a specific evaluator, compiler, solver or device. Evidence records what was established about that realization and its outputs. These objects are connected by explicit dependencies rather than merged into a single status field.

The proposal develops from an existing implementation. In BiDi, CDC connects canonical source to native state transitions, guarded effects and receipts; U1 adds lifted-cover acceptance and U2 adds variation followed by recurrence-qualified return analysis [3]. U abstracts a broader composition discipline after that work. CDC predates U and provides an independent reference against which the corresponding U operations can be compared.

The contribution is an integration protocol and a concrete proof program built on typed graphs, algebraic effects, linear resources and categorical composition. The paper develops four connected results:

1. a structural calculus whose admitted profiles retain domain-specific law premises and whose derived operations remain inspectable;
2. compositional obligations for resource independence and cross-theory adapters, including explicit assumption and approximation accounting;
3. identities and evidence rules that distinguish a represented meaning, its implementation, its execution and its earned consequences; and
4. a compatibility case study that retains CDC’s actual finite maps, derivatives, discrete modes and recurrence refusal.

The ambition is a useful general programming language and semantic substrate. The design excludes an exact terminating evaluator for every admitted mathematical description, an automatic equivalence decision for arbitrary source languages, and the inference of physical execution from simulation. Those exclusions follow from the meaning of the proposed contracts; they are not temporary performance limitations.

1.1 Three discriminating examples

Copying. A reusable integer admits a diagonal operation $x \mapsto (x, x)$. An owned resource does not acquire a second owner because its handle appears twice. Unknown quantum data does not acquire a general copy operation because its circuit description is classical data. Wiring must therefore lack implicit contraction.

Observation order. Let C be a fair coin. A decision b made without observing C satisfies $\Pr[b = C] = 1/2$, including randomized choices independent of C . If the decision observes C , the policy $b = C$ achieves probability one. These programs use the same ingredients with different information access. A transformation that interchanges probability and choice without preserving the observation policy is unsound.

Finite time steps. In a scalar instance of the pinned CDC flow map, $F_d(x) = x + d \cos x$. Consequently,

$$F_1(F_1(0)) = 1 + \cos 1 \neq 2 = F_2(0).$$

The exact-flow semigroup law is not available for this realization. A U import that replaces it with an ideal ODE has changed the operation that CDC actually executes.

2 Semantic objects and admission

Definition 2.1 (Semantic profile). *A profile T specifies a class of interfaces $\mathcal{I}_T$, a class of operation meanings $\mathcal{M}_T(A, B)$ for admissible boundaries, an observation family $\mathcal{O}_T$, a signature of generators, their laws and side conditions, numeric and temporal conventions, and its assumptions. It also specifies which structural constructors it interprets and which compositions are defined.*

This definition does not require every model to be a set-theoretic total function. A profile may use partial maps, relations, measures, behavior sets, path spaces or quantum instruments. It must state its observation relation explicitly. Two profiles can interpret an apparent common carrier using different observables; a shared type name alone does not identify them.

Definition 2.2 (Operation descriptor). *A generator descriptor is a tuple*

$$o = (A, B, T, \epsilon, u, s, \mathcal{L}, r, \mathcal{O}, \mathcal{A}, \mathcal{D}),$$

where A, B are interfaces, ϵ is the effect interface, u the resource-usage account, s a stage, $\mathcal{L}$ the applicable laws and their premises, r a direct rule or checked expansion, $\mathcal{O}$ the observations, $\mathcal{A}$ the assumptions and $\mathcal{D}$ immutable dependencies. The descriptor also records failure and hold conditions and the trusted implementation/checker boundary.

The distinction between direct and derived operations is accountable, not cosmetic. A derived SQL join retains a visible join identity and a checked expansion dependency; lowering it to smaller operations need not erase that identity. Conversely, naming a foreign callback `join` does not establish a U relational semantics.

2.1 Admission is a product of dimensions

For an operation o , an admission record has the shape

$$\text{Adm}(o) = (\text{origin}, \text{support}, \text{laws}, \text{assumptions}, \text{trust}).$$

Origin can identify a primitive, derived definition, library operation or foreign import. Support is indexed by realization: reference evaluation, compiled code, simulator or actual device. Laws identify particular checked statements, not a blanket proof badge. Trust lists checker and implementation dependencies. An admitted primitive can have a checked law and an unresolved assumption while lacking a hardware backend. Collapsing these fields into one exclusive enumeration loses information needed to judge a composition.

The initial seven families, V/M/R/P/C/D/Q, organize this accounting. They do not assert seven irreducible axioms. The cost of generators, type rules, models, search strategies, solver assumptions and trusted code must remain visible beneath each family. Structural brevity is a separate measurement from semantic, implementation and human comprehension costs.

2.2 Configuration and outcomes

A reference configuration may be written

$$K = (G, \eta, S, \Delta, E, \kappa, \Xi),$$

with graph G , bindings η , stores S , resource ledger Δ , event structure E , continuation κ and evidence context Ξ . A store is an indexed carrier appropriate to its profile, not a promise that physical quantum state or every continuous trajectory is a classical serializable heap value.

An outcome distinguishes successful completion, semantic refusal, invalid input, execution failure and

uncertainty about an external effect:

$$\begin{aligned}
\text{Outcome}(A) = & \text{Done}(A, \text{receipt}) \\
& + \text{Held}(\text{obligations}, \text{earned artifacts}) \\
& + \text{Rejected}(\text{diagnostic}) \\
& + \text{Fault}(\text{error}, \text{known progress}) \\
& + \text{Indeterminate}(\text{recovery obligation}, \text{known progress}).
\end{aligned}$$

A hold protects the state designated by the refused operation. It need not roll back earlier effects, remove an audit event or restore consumed attempt identities. A budget interruption does not decide mathematical divergence.

3 Structural calculus and deterministic elaboration

For admitted theory environment Θ , the structural syntax is

$$\begin{aligned}
p ::= & \text{wire}[\pi] \mid \text{gen}[T, o](\vec{a}; \vec{r}) \mid \text{seq}(p, p) \mid \text{par}(p, p) \\
& \mid \text{scope}(n, p) \mid \text{fix}[m](x, p).
\end{aligned}$$

Here π permutes compatible ports; $\vec{a}$ are static arguments; $\vec{r}$ are typed, staged regions; n is a nominal binder; and m names an admitted iteration discipline. This presentation is economical, not a proof that no five-tag presentation could encode the same information.

Use the judgment

$$\Theta; \Gamma; \Delta \vdash p : A \xrightarrow{T, \epsilon, u, s} B.$$

Γ contains reusable bindings and Δ affine or linear resources. Sequential composition connects the same boundary under the same admitted profile; a mixed boundary is first mediated by an explicit adapter. Parallel composition requires an independent-resource judgment. Scope binds names without capture or inadmissible escape. Iteration is qualified: partial computation, monotone relational iteration, clock-guarded feedback and total structural recursion have different premises.

3.1 Local obligations imply a structural theorem

Fix one admitted profile and its environment. Assume:

1. each generator reduction preserves its declared interface and usage;
2. wiring is a type-preserving permutation with no copying or dropping;
3. serial reduction composes only matching interfaces;
4. parallel reduction respects its valid split and does not modify the other branch's exclusive resources;
5. substitution is capture-avoiding and preserves typing and usage; and
6. the selected iteration discipline admits its unfolding or feedback rule with the same input/output contract.

These are admission obligations on a profile, not conclusions of graph shape alone. In particular, a generator with an incorrect local rule defeats the theorem's assumptions.

Theorem 3.1 (Conditional structural interface preservation). *Under the six assumptions above, if p has boundary $A \rightarrow B$ and resource account Δ , a well-defined structural reduction of p preserves that boundary and the profile's resource invariant, up to the explicitly admitted consumption or creation of resources in generator rules.*

Proof. Induct on the derivation of the reduction. A generator step uses assumption 1; a wiring reduction uses assumption 2. A left serial-context step changes only its subderivation, so its output remains the unchanged right operation's input by induction and assumption 3. The right-context case is symmetric after the left result has been obtained. A parallel-context step preserves its local boundary by induction and leaves the

other resource partition untouched by assumption 4, so the tensor boundary and split are retained. Scope reduction uses freshness and assumption 5 to prevent capture or escape. Iteration unfolding is a permitted substitution with the same boundary by assumptions 5 and 6. These cases exhaust the structural rules; none permits an unadmitted generator or a boundary cast. $\square$

Scope 3.2. This is a compositional metatheorem. It is neither a complete subject-reduction proof for every proposed U domain nor a proof of the complete native implementation. Progress additionally requires each active generator to make a valid step, return a permitted outcome, or be allowed to wait. Event waits, partial recursion and unsupported solvers preclude a universal termination theorem.

3.2 Whole-surface elaboration

Let s be a parsed source tree, M a fully resolved immutable manifest, and v an elaboration version. Define

$$\mathcal{E}_v(s, M) = (G, B, W, U),$$

where G is the graph, B explicit bindings, W inferred adapter and profile evidence, and U unresolved obligations. Source definitions, records, calls, lets, regions, staged syntax and derived operations all need versioned rules. A parser accepting their spelling is insufficient.

Proposition 3.3 (Deterministic elaboration contract). *If the parser, name resolution, overload selection, traversal order, binder naming, adapter selection, expansion lookup and serialization are deterministic functions of (s, M, v) , repeated successful elaboration yields identical canonical bytes.*

Proof. Induct over the finite syntax tree. The leaf rules choose identical resolved identities. At an internal node, induction gives identical elaborated children; deterministic rule selection, binder construction and witness generation then give identical parent objects. The root objects are equal, and deterministic canonical serialization preserves equality. $\square$

The premises explicitly exclude environment-dependent lookup, mutable package aliases, unordered iteration with observable serialization, and selection among ambiguous adapters without a rule. A fresh-name counter must derive from deterministic traversal rather than an ambient process clock. The result is deterministic representation, not uniqueness up to all semantic equivalences. An implementation must test this proposition's concrete premises rather than cite the proposition as evidence they were implemented.

4 Resources and conservative independence

Let $\mathcal{R}$ be a finite set of nominal resource identities. A footprint for operation p consists of owned resources O_p , reads R_p , writes W_p and an effect/observation contract. Read and write sets denote actual resource identities after alias resolution. A read may use an owned resource or a separately admitted shared capability. Unknown aliasing produces an obligation rather than an assumption of disjointness.

Definition 4.1 (First independence witness). *For explicit shared-read set S , define $I_S(p, q)$ by*

$$\begin{aligned} O_p \cap O_q &= \emptyset, \\ W_p \cap (R_q \cup W_q) &= \emptyset, \\ W_q \cap (R_p \cup W_p) &= \emptyset, \\ R_p \cap R_q &\subseteq S. \end{aligned}$$

The witness is valid only if its identities, footprints and read permissions are checked and the composite theory profile is admitted.

Theorem 4.2 (Owned-resource nonduplication in the finite model). *Suppose the initial ledger assigns each linear resource at most one owner, every allocation creates a fresh identity, ownership transfer consumes the old owning permission, and every parallel split satisfies I_S . No finite execution composed of those steps contains two live owning permissions for one linear resource.*

Proof. Induct on the number of allocation, transfer, consumption and split steps. The initial property holds by assumption. Allocation adds an owner of an identity absent from the ledger. Transfer removes the old permission before inserting its replacement. Consumption removes a permission. A parallel split partitions owned identities into disjoint sets, so an identity cannot be assigned to both branches; shared-read witnesses add no owner. Joining the partitions preserves disjointness because each branch is constrained by the same rules and fresh allocation. These cases preserve the invariant. $\square$

Scope 4.3. The model trusts identity freshness and footprint correctness. It does not prove a hardware memory model, arbitrary alias analysis, distributed lease freshness or physical no-cloning. An implementation whose user can forge owning handles violates the premises. A read capability and an authority lease remain different contracts even when both have a scope.

Lemma 4.4 (Qualified commuting updates). *Let p, q be deterministic terminating store transformations with correct finite footprints. Suppose their reads include every store dependency, their writes contain every changed location, $I_S(p, q)$ holds, allocation and external effects are absent, and observations compare final stores and branch results without exposing order. Then executing $p; q$ and $q; p$ produces the same observed result.*

Proof. Neither transformation changes any value read by the other, so determinism gives the same branch result and proposed writes in either order. Their write sets are disjoint, hence applying those updates in either order yields the same final store. The stipulated observation ignores execution order. $\square$

This lemma explains why disjoint writes can support one particular optimization. It does not license the same rewrite when an observer sees event order, operations allocate nominal resources, time matters, or a device has unmodeled shared state. Named causal edges generally establish dependence, not the independence certificate needed by the lemma.

5 Adapters, assumptions and approximation

An adapter connects profiles through a declared relation; it need not assert an isomorphism. Typical boundaries include quantum measurement into a classical record, a probability measure into a sampled trace, a differential equation into a numerical trajectory, a store snapshot into a relational bag, and a CDC phase into a balanced-ternary observation.

Definition 5.1 (Relational adapter). *An adapter $\alpha : T \rightsquigarrow S$ consists of an interface map, a translation on its admitted fragment, a relation $\mathcal{R}_\alpha \subseteq \mathcal{O}_T \times \mathcal{O}_S$, assumptions A_α , a checker or explicitly trusted argument, a loss account and a residual policy. Its preservation assertion says that every admitted translated execution has the stated related source/target observations, under A_α and the chosen quantifier direction.*

Quantifier direction matters. Equality of all behavior sets, simulation, trace refinement, one paired execution and statistical comparison are different statements. A deterministic scheduler may refine permitted nondeterminism while failing to preserve all source schedules. An adapter must not report that refinement as equivalence.

Theorem 5.2 (Conditional relational composition). *Suppose $\alpha : T \rightsquigarrow S$ and $\beta : S \rightsquigarrow V$ agree on the intermediate interface, observation semantics and translated object. Suppose their preservation assertions apply to that same intermediate execution, with compatible quantifier directions. Under $A_\alpha \cup A_\beta$, the composite has preservation relation*

$$\mathcal{R}_{\beta\alpha} = \{(x, z) \mid \exists y. (x, y) \in \mathcal{R}_\alpha \wedge (y, z) \in \mathcal{R}_\beta\}.$$

Proof. For any admitted composite execution, the second preservation assertion provides its intermediate observation y related to final observation z . Compatibility makes this the intermediate object governed by the first assertion, which provides x with $(x, y) \in \mathcal{R}_\alpha$. Therefore y witnesses membership of (x, z) in the displayed relation. Both preservation uses retain their premises, so the combined assertion depends on their union. No assumption is discharged by composition alone. $\square$

The theorem is deliberately conditional on a compatible intermediate observation and quantifiers. Two isolated success receipts cannot prove that compatibility. If one adapter hides information on which the second depends, a new contract is required. Likewise, reducing the written assumption set needs a checked implication or proof discharge; deleting a field from metadata does not strengthen a theorem.

Proposition 5.3 (A quantitative special case). *Let $f : X \rightarrow Y$ and $g : Y \rightarrow Z$ be ideal maps between metric spaces, with realizations $\hat{f}, \hat{g}$. On an admitted domain suppose*

$$d_Y(\hat{f}(x), f(x)) \leq \epsilon_f, \quad d_Z(\hat{g}(y), g(y)) \leq \epsilon_g,$$

and g is L_g -Lipschitz on a domain containing $f(x)$ and $\hat{f}(x)$. Then

$$d_Z(\hat{g}(\hat{f}(x)), g(f(x))) \leq \epsilon_g + L_g \epsilon_f.$$

Proof. Insert $g(\hat{f}(x))$ and use the triangle inequality. The first distance is at most ϵ_g by the uniform realization guarantee at $\hat{f}(x)$; the second is at most $L_g \epsilon_f$ by Lipschitz continuity. $\square$

A tolerance slider supplies neither a uniform error bound nor the required domain containment and Lipschitz constant. This proposition therefore gives an admissible composition rule when those facts are available, and an explicit obligation when they are not. Stochastic, hybrid and relational adapters may need different metrics and preservation relations.

6 A total proof boundary

U's design permits programs to search, diverge, use IO and request solvers. Its logical conversion relation must have a smaller boundary. Proof terms may share some syntax with programs without inheriting unrestricted evaluation rights. A tactic can return a candidate whose acceptance is independently checked; a successful host callback cannot construct the authority-bearing result merely by returning its name.

Consider a small illustrative arithmetic fragment with natural numbers, successor, addition recursing on the second argument, equality, reflexivity, congruence and induction. Its defining equations are

$$\text{add}(a, 0) = a, \quad \text{add}(a, Sb) = S(\text{add}(a, b)).$$

For concrete natural inputs, the recursive evaluation decreases the second argument on each addition call. Equality proofs use these equations and the declared inference rules; general partial fixed points are absent.

Proposition 6.1 (An inductive witness). *For every $n \in \mathbb{N}$, $\text{add}(0, n) = n$.*

Proof. At zero, the left side reduces to zero. Assume $\text{add}(0, k) = k$. By the successor equation and congruence, $\text{add}(0, Sk) = S(\text{add}(0, k)) = Sk$. Natural-number induction concludes the statement. $\square$

This example is intentionally not reflexivity at an arbitrary symbolic n : addition reduces on its second argument, so the universal statement needs induction. A checker that treats the spelling of a proposed proof as sufficient has skipped the central operation.

Proposition 6.2 (Constructor-bounded proof authority). *Suppose the only way to create an accepted proof artifact is a checker that recursively validates a finite derivation using a fixed set of sound rules, with conversion performed solely by its terminating admitted conversion procedure. Suppose its proof-artifact constructor is inaccessible to untrusted programs. Then every artifact it issues has a valid derivation under its recorded assumptions; a partial computation's result alone is insufficient to obtain one.*

Proof. Induct on the finite checked derivation. Each leaf is a valid axiom or a recorded assumption. For an internal node, induction establishes its premises and the sound checked rule establishes the conclusion. The terminating conversion procedure only permits its declared equalities. Constructor exclusivity excludes creation outside this validation path. $\square$

The theorem does not independently establish soundness or termination of an arbitrary extensible dependent type theory. Those are premises to discharge for each actual checker. U implements the admitted Nat/Pi/Eq fragment as a separate kernel and checks the preserved induction example through that kernel. Its native code, representation guards and execution environment remain in the trusted implementation boundary. These checks do not establish adversarial process isolation, full universes, positivity, general dependent elimination or metatheoretic correspondence for the complete language.

7 Identity, dependencies and invalidation

An artifact needs more than one identity. Original-source identity binds exact bytes and the source environment. Canonical-syntax identity binds normalized syntax and its version. Meaning identity binds graph, profiles, numeric and observation conventions, expansion dependencies and assumptions. Realization identity binds implementation. Execution identity also binds input and relevant scheduler, randomness or external decisions. Replay identity may select a declared projection of history. Resource identity is nominal and can include an epoch; equal contents do not imply equal resources.

Definition 7.1 (Canonical dependency identity). *Let D be a finite directed acyclic graph of canonical objects, whose edges point to immutable dependencies. For object v , define*

$$h(v) = H(\text{canon}(\text{tag}(v), \text{payload}(v), [h(w)]_{w \in \text{dep}(v)})),$$

with an explicit dependency ordering or canonical set ordering. Recursive syntax is represented by binders and references within finite objects, not by circular content-hash equations.

The canonical encoding must be unambiguous for its domain. Numeric values need a declared encoding, including finite IEEE bit patterns where relevant, and an explicit policy for signed zero, nonfinite values and mathematical integers. A serializer that conflates these values cannot preserve a profile that distinguishes them. Human-readable receipt rounding can coexist with an exact-bit sidecar; it cannot silently redefine legacy receipt identity.

Lemma 7.2 (Representation identity and backend separation). *If two builds have identical canonical meaning objects and dependencies, they have equal meaning digests. A realization change outside those objects can preserve the meaning digest while changing realization identity.*

Proof. Evaluate the DAG in a topological order. Equal leaf encodings have equal digests; identical parent encodings and child digests then give equal parent digests. The realization's encoding is a distinct object, so changing it does not alter the fixed meaning object. $\square$

The premise includes the whole semantic contract. Changing precision, rounding, observation order or time behavior that the profile exposes is a meaning-object change, even if described informally as a backend swap. Meaning identity is identity of a representation and contract; it is not a decision procedure for arbitrary extensional equivalence.

Lemma 7.3 (Dependency invalidation under a finite collision assumption). *Assume the encoding is injective on encountered objects and the digest has no collisions among their encodings. If an object's payload or a referenced dependency changes, its digest changes, and the change propagates to every ancestor whose canonical object retains that dependency. A receipt bound to the old root cannot validate the new root by digest equality.*

Proof. At the changed object, injective encoding gives different bytes and the finite collision assumption gives different digests. At each retained parent edge, the changed child digest changes the canonical parent bytes; the same reasoning changes the parent digest. Induct on distance to each ancestor. The new root digest differs, so equality with the receipt’s old subject fails. $\square$

This lemma assumes dependency completeness. Omitting a law expansion, numeric profile or external contract from the dependency graph can leave an invalid receipt apparently current. Cryptographic collision resistance does not repair that modeling error. A digest also provides neither sender authentication nor truthfulness. Signed statements can be false, and a commitment to discarded history cannot reconstruct that history.

8 Earned artifacts and guarded consequences

For a fixed semantic subject s , let $\mathcal{A}_s$ be a set of possible artifact kinds with a prerequisite relation. For example, an accepted path may precede a tangent, a tangent plus recurrence may precede monodromy, and monodromy plus a valid spectrum computation may precede multipliers. The prerequisite relation is acyclic within a finite admission decision.

Definition 8.1 (Evidence transition). *A state is (s, E, O) , with earned artifact set $E \subseteq \mathcal{A}_s$ and outstanding obligations O . An earning step may add a only when a trusted constructor validates its subject identity, every prerequisite is in E , and its own gate succeeds. A held step retains E and records the unsatisfied obligation. Invalidation for a changed subject creates a new subject context; it does not relabel old evidence as evidence of the new one.*

Theorem 8.2 (Monotonic earning and non-fabrication). *For finite transitions governed by the definition, earned sets are monotone within one fixed subject. Every earned artifact has an earned prerequisite chain and a successful own gate. If an indispensable prerequisite is never earned, no artifact requiring it can be earned.*

Proof. Induct on transitions. A held step retains the set. An earning step adds a single artifact and removes none, proving monotonicity. Its constructor checks all prerequisites and its own gate; induction supplies the valid chains behind those prerequisites. If a required predecessor is absent, the insertion precondition is false, preventing that artifact and, by the same argument along the dependency chain, its dependents. $\square$

The set model captures a guarantee about what is authorized, not a guarantee that every gate implementation is correct or every observation truthful. Those remain trusted dependencies. Artifacts can also expire or become inadmissible for a new request; that is a new validity decision, not a retroactive claim that a historical computation never happened.

The CDC example shows why an ordered Boolean success flag is inadequate. If recurrence fails, a valid tangent remains earned but monodromy and multipliers are absent. If recurrence and monodromy pass and a spectral backend is unavailable, monodromy remains earned while multipliers are held. A downstream refusal neither promotes weaker evidence nor destroys an independently valid earlier artifact.

8.1 Effects and durability

An admitted external effect adds obligations beyond the abstract artifact rule: capability scope, freshness, causal parent, replay protection and failure atomicity appropriate to that effect. Memory atomicity, a local durable transaction and a distributed command are not one universal primitive. A timeout after sending a command can leave completion indeterminate. Exactly-once external action does not follow from deterministic local replay.

This distinction follows the pinned BiDi toolchain and RFTC contracts. Local journal replay and authenticated envelopes have particular mechanisms and counterexamples; their presence does not establish a deployed multi-host protocol or a physical law. U preserves these as distinct implementation gates rather than inheriting their strongest possible interpretation.

9 Residuals and native semantic possession

An import must declare a source fragment and an observational relation. For source bytes P , define

$$\text{lift}(P) = (G, R, \Pi),$$

where G is a typed U graph, R a residual and Π provenance. Residuals may retain lexical layout, identifiers and intensional structure, unmodeled semantic regions, environment assumptions and approximation data. Retaining a source region does not imply U can execute or analyze it natively.

Definition 9.1 (Exact reconstruction). *A lift/reconstruction pair is exact on fragment F if for every $P \in F$,*

$$\text{reconstruct}(\text{lift}(P)) = P$$

as bytes. Semantic preservation is a separate judgment relating the source and target observation sets under their named profiles.

Proposition 9.2 (Reconstruction is insufficient for possession). *Exact reconstruction alone entails no semantic interpretation of the source.*

Proof. Define a lift that maps every P to one constant graph and stores the whole P in its residual. Reconstruction returns that residual. The equation holds for every source, although the graph contains no distinctions among their behaviors. Thus the equation cannot imply semantic interpretation. $\square$

For structured edits, let $g : S \rightarrow V$ extract a view and $p : V \times S \rightarrow S$ put an updated view into an old source. On an admitted domain, the familiar lens laws are

$$p(g(s), s) = s, \quad g(p(v, s)) = v.$$

They connect preservation of unchanged source with faithful propagation of a changed view; bidirectional transformation work supplies the relevant background [4].

Proposition 9.3 (Scoped reconstruction after edits). *If g, p satisfy the two displayed laws on their admitted source/view domain, an unchanged view reconstructs the same source, and putting a valid changed view yields a source whose extracted view is that change. If semantic interpretation factors through g on that domain, the reconstructed source has the changed view's semantics.*

Proof. The first two conclusions are the two laws instantiated at s, v . For the last conclusion, write source interpretation as $J \circ g$; then $J(g(p(v, s))) = J(v)$ by the second law. $\square$

Factoring through the view is a substantial assumption. If behavior depends on residual content omitted from the view, the final conclusion is unavailable. A transformation without a valid residual-update rule must invalidate exact export or return a narrower guarantee. Reattaching stale comments or old tokens to a changed graph cannot certify reconstruction.

9.1 A fragment-indexed possession criterion

For an admitted source fragment, native possession requires six kinds of evidence: explicit target operators; compositional meanings; direct U rules or checked expansions; analysis of those operators; recoverability with declared loss; and differential/adversarial preservation checks. A hidden source-language interpreter behind one generic node fails this criterion even when it computes the correct output. Turing completeness says nothing about the representation preserving source laws in an inspectable form.

Different source fragments need different relations. C and assembly need memory/ABI and defined-behavior boundaries; Haskell needs laziness and bottom; SQL needs bags, NULL and collation; temporal models need behavior and fairness; probabilistic programs need distributions and inference assumptions; hardware needs cycle and event-region semantics; quantum programs need joint-channel and measurement semantics. No one test transcript proves all of these relations.

10 CDC specialization and compatibility

The compatibility reference is BiDi commit `1307f2a7f32ab5beb64fe5cd0c0faf13ec9c8157`, release 0.3.0, grammar 1 and C ABI 1.5. Its native implementation and source contracts remain independent authority. U’s later realization must preserve their observable maps before claiming parity. Historical CI and witness counts reported by that repository are historical evidence, not fresh U test results.

The three primitive native reductions are flow, commit and nest. Frameworks, U1, U2, persistence and authority are derived protocols. The richer formal spine may specify amplitude, delays, plasticity, localized events or projected relations that the native realization does not consume as dynamics. Unconsumed declarations remain obligations rather than extra silently executed coordinates.

10.1 Synchronous flow and its actual Jacobian

For selected cell i , native flow uses one pre-step snapshot:

$$\theta'_i = \theta_i + \omega_i d + Gd \sum_{e:t(e)=i} w_e \sin(\theta_{s(e)} + \alpha_e - \theta_i). \quad (1)$$

Only channels admitted by the pinned field/profile contract contribute. Parameters are frozen for this derivative. Direct differentiation gives

$$\frac{\partial \theta'_i}{\partial \theta_\ell} = \delta_{i\ell} + Gd \sum_{e:t(e)=i} w_e \cos(\theta_{s(e)} + \alpha_e - \theta_i) (\delta_{s(e)\ell} - \delta_{i\ell}). \quad (2)$$

The analytic derivative describes the real extension of the finite-step expression in Eq. (1), not the exact solution map of a continuous equation or a derivative of literal IEEE rounding. In particular, a self-channel’s phase differences have the cancellation expressed by the two Kronecker terms. Reversing or sequentializing the pre-step reads changes the map and its Jacobian.

The numeric realization has an additional exact-bit contract. In the inspected arm64 Apple Clang 21 -02 oracle, generated fused multiply-add instructions make some expressions observably different from separate multiplication and addition. The U compatibility path therefore uses explicit `math.fma` operations from U at the corresponding sites. The native bridge realizes those operations through the platform’s fused arithmetic primitive; the independent Python reference uses its own explicit FMA path. Compiler flags and contraction policy belong in the realization/profile record. Matching binary64 storage alone is insufficient to establish bitwise parity. Analytic finite-map differentiation and numerical agreement with that pinned implementation are separate claims; the former does not differentiate the discontinuities of rounded machine arithmetic.

10.2 Commit and nest in a fixed discrete itinerary

With deadband δ , native quantization is

$$\tau(\theta) = \begin{cases} +1 & \cos \theta > \delta, \\ -1 & \cos \theta < -\delta, \\ 0 & \text{otherwise.} \end{cases}$$

An ordered trit sequence is accepted only when every prefix sum is nonnegative. A scheduled accepted commit writes latch state while leaving the continuous coordinates unchanged. Its continuous reset derivative is identity within a fixed accepted itinerary. At a quantization boundary or when perturbation changes the discrete itinerary, that local derivative does not establish a derivative across the branch boundary.

Let $u(q)$ be the child mean trit under the pinned latch/fallback rule. Within fixed discrete mode q , nest executes

$$b'_p = b_p + Gu(q), \quad p'_c = b'_p.$$

The new child prior depends on the old parent belief, not on its own old prior. With respect to (b_p, p_c) the affected Jacobian block is

$$\begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}.$$

The zeros assume fixed trits and frozen gain. A derivative of a richer theoretical feedback model is not a substitute for the derivative of this assignment.

10.3 Scheduled events versus localized guards

If an event is scheduled by the source, the reset derivative is DR under that schedule. A state-localized transverse guard may instead require saltation. Under differentiability, fixed itinerary, guard $g(t, x) = 0$ with $n = \nabla_x g$, reset $R(t, x)$ and nonzero denominator, the saltation expression has the form

$$S = DR + \frac{(f^+ - DRf^- - \partial_t R)n^\top}{n^\top f^- + \partial_t g}.$$

The localized event contract must establish the actual guard, crossing, vector fields and reset. A scheduled commit does not become such a localized event because a saltation formula is available. Grazing, missing event localization or altered order blocks the corresponding derivative claim. The pinned U2 contract makes these distinctions explicit [3].

10.4 Tangent, recurrence and return

Let the accepted finite path be $F = F_n \circ \dots \circ F_1$ on a source-bound coordinate manifest. For column tangents, initialize $A_0 = I$ and update

$$A_k = J_k A_{k-1}, \quad A_n = J_n \dots J_1.$$

The ordinary chain rule establishes this derivative wherever the admitted local differentiability and fixed-itinerary conditions hold. Reversing the product is generally wrong.

Absolute recurrence requires the full declared continuous endpoint to return, together with discrete-mode restoration. Relative recurrence requires an explicit restoration ρ , its derivative, its domain and the requisite symmetry/equivariance contract. Only then is the relative return derivative

$$M_{\text{rel}} = D\rho(F(x_0))A_n.$$

A projected phase match cannot certify complete return. A matrix can have eigenvalues without those numbers being Floquet multipliers of a periodic orbit. Numerical tolerance creates a profile-qualified recurrence test; it is not an exact periodic-orbit theorem without further error analysis.

Corollary 10.1 (Recurrence-gated non-fabrication). *If the evidence gate for monodromy requires a valid tangent and verified full or admitted relative recurrence, then a recurrence hold prevents monodromy and all dependent spectrum artifacts while retaining the valid tangent. If monodromy is already earned and a spectral backend holds, the monodromy remains earned.*

Proof. Apply the prerequisite and monotonicity theorem of Section 8 to the tangent/recurrence/monodromy/spectrum graph. $\square$

The pinned canonical two-turn loop illustrates the first case: lifted-cover U1 closure and a tangent do not imply full-state return when unwrapped phase, latch presence, belief and prior fail the return contract. The separate relative-return calibration declares a 4π endpoint restoration and its complete derivative. It is a different fixture, not a replacement interpretation of the canonical loop. U must preserve both distinctions.

10.5 Framework, toolchain and RFTC boundaries

BiDi’s frameworks attach role contracts and executed exemplars to existing machinery. U should retain their derived identities and checked bindings, rather than counting every task-shaped name as a new universal primitive. The toolchain’s binding amendments distinguish canonical source parsing, ABI ownership, content digests, keyed authority, crash durability, per-check parity and trusted-local package execution.

BiDi’s earlier native runtime remains an independent implementation. Compatibility is assessed through explicit source, state, numeric and observation contracts. RFTC supplies local classical synchronization and record-closure mechanisms; distributed and physical realizations require separate evidence. A U integration must identify and exercise the specific contract at each boundary.

11 Native implementation and verification

The compiler is a U program. It owns scanning, parsing, lexical and core annotation checks, closure conversion and deterministic native-code emission. A second U module lowers executable bodies into the six structural constructors and reconstructs the program from graph regions and bindings. Declared types remain interface metadata; no generator stores an executable body as an opaque syntax tree. The command-line path compiles the reconstructed program with the selected U libraries.

11.1 Source, graph and native execution

The graph retains qualified operator names, nominal binding identities, ordered operands, stages, imported profile identifiers and resource-use obligations. Reconstruction checks tags, arities, scopes, literal encodings and canonical metadata. Literal-only operand groups have an explicit empty resource-footprint witness; other operand groups remain ordered. This local structural rule does not establish independence for arbitrary effectful branches.

All 22 preserved examples reconstruct to their native-parser ASTs and agree with the independent reference parser after accounting for their different coordinate and record-field encodings. Repeated lowering is deterministic; source-offset changes leave the structural identity unchanged. Corrupted tags, arities, binders, stages, operator identities, parallel witnesses and literal encodings are rejected even when the enclosing digest is recomputed. Source bytes are auxiliary provenance. Replacing them cannot replace the executable regions reconstructed from the graph.

The resulting C contains native functions for U functions and closures. Generated C is a portable machine realization; U owns the algorithms that construct it. A handwritten bridge supplies generic values, arbitrary-precision integers, storage, function application and platform access. TLS and SHA-256 use declared platform implementations. The bridge does not parse or interpret a U program or implement the CDC, inference or proof algorithms. Its default allocation arena is bounded at 1 GiB and released at process exit. That budget is neither a tracing collector nor a bound on total operating-system memory use.

The formatter is also a U program. It preserves comment, string and numeric lexemes, reparses its output and is idempotent on the verified source corpus. A separate U-written backend performs interval analysis and emits WebAssembly bytes directly for admitted integer expressions and Boolean predicates. Every intermediate must fit its certified signed-i64 interval. The host must check exact integer inputs before ABI conversion; an in-module guard cannot recover bits already truncated by the host. This target is narrower than the general native compiler.

11.2 Executable library scope

Table 1 summarizes the current U-owned responsibilities. Support remains a relation between an operation, source fragment, semantic profile and realization; the table does not turn a family name into complete domain support.

HMC preserves its seeded random stream and Markov state across bounded, unprivileged worker batches.

Table 1: Native U responsibilities and their present boundaries.

Area	Implemented in U	Boundary
Values and syntax	Collections, natural recursion, lexical closures, quotation, splicing and single-evaluation binding	General staged-type and resource theorems require separate correspondence.
Relations and probability	Occurs-checked unification, relational search, bag joins, weighted models, conditioning and seeded HMC	Declared search and numerical inference policies; no convergence or arbitrary import claim.
Behavior	Local tasks, cancellation, HTTP operations, actor supervision and finite temporal checks	Local execution does not establish distributed or all-schedule correctness.
Models	Array operations, affine index-1 backward Euler, clocked-register and partitioned-launch simulation, joint statevectors	Numerical and simulated realizations retain their declared scope.
Proof and evidence	Total Nat/Pi/Eq checking, subject identities, checked handles and guarded consequences	No complete dependent type theory or whole-runtime correctness theorem.
CDC	Finite reductions, executed paths, ordered tangents, recurrence, return operators and triangular spectra	Full U1 binding, localized events and general dense Schur analysis remain open.

The original four-chain workload completes with 1,000 retained draws and 500 warmup transitions per chain; a shorter execution checks exact agreement with the unbatched stream. Those results establish execution and reproducibility, not statistical convergence.

The CDC analysis issues opaque handles through a private U registry. Matching a label or constructing a new generic resource cannot insert an entry into that registry. Tangent, recurrence and return constructors check the same live path identity, canonical input binding and coordinate manifest. The source identity supplied by a caller is a bound label, not an authenticated claim about an external file.

Relative restoration currently supports explicitly declared uniform integer phase turns on coupled state. Its restoration derivative is constructed as identity. The certificate records numerical endpoint agreement and the periodic real-map interpretation; it does not assert exact global IEEE-arithmetic equivariance. Upper or lower triangular return matrices have a validated real-Schur path with reconstruction and orthogonality residuals. A general dense matrix produces a hold while retaining the checked return operator. The complete U1 role, cover, decision and effect bindings are required separately and cannot be supplied as caller Boolean certificates.

11.3 Reproducible self-compilation

A checked generated-C seed establishes the initial executable. Subsequent translation is performed by the U compiler. The reproduction procedure compiles the compiler, uses that executable to compile the same U source again, and repeats the process through a third stage. Generated C agrees byte-for-byte across the tested stages. Native executables also agree under the recorded toolchain when built with the same output basename, which matters for macOS ad-hoc signature identity.

Compiler and program tests run with no interpreter in their executable search path. The independent historical Python implementation and optional seed translator remain comparison tools; ordinary compilation and execution do not invoke them. Self-compilation establishes a reproducible translator fixed point. It does not by itself exclude a compromised seed, prove the C toolchain correct or discharge a domain’s mathematical laws.

11.4 Discriminating checks

The verification program follows mechanisms and refusal conditions:

- elaborate repeated input with the same manifest to equal canonical bytes; change an expansion dependency and invalidate dependent evidence;
- distinguish equal-named and equal-content resources from identical allocations; reject duplicate ownership, shared writers and escaped views;
- preserve the coin/choice observation-order difference and SQL bag multiplicities; hold undefined normalization instead of inventing a posterior;
- prevent partial recursion, unsafe coercion and external success strings from producing accepted proof objects;
- compare direct evaluation with each admitted compiled fragment under the same overflow, ordering and numeric contract;
- compare CDC state, decisions, effects and derivative products to the pinned independent implementation; reject false projected recurrence;
- retain already earned artifacts when a later gate holds, while rejecting stale subject identity or fabricated prerequisite records; and
- reconstruct original source bytes where promised, and refuse exact export after edits that lack a valid residual update.

These acceptance criteria test different aspects of the implementation. Unit, property, differential, mutation and security checks have different scopes. A mutation family is tied to an actual mechanism: reversed Jacobian order, wrong nest derivative, omitted restoration derivative, false recurrence, or a stale capability receipt. Passing a finite sample is evidence about that sample and mechanism; it is not an unbounded theorem.

11.5 Reproduction and provenance

With a C11 toolchain installed, the public launcher compiles and runs U programs through the native path:

```
./bin/etellis-u run examples/original/02_arithmetic.u \  
  --entry square --args '[7]' \  
./bin/etellis-u prove examples/original/14_proof.u \  
  --entry checked
```

The arithmetic example returns 49. A Fibonacci program compiled through the graph returns the exact integer 354224848179261915075 at input 100. The proof example is checked by the U Nat/Pi/Eq kernel rather than accepted from its annotation. Build records identify source, graph, compiler, runtime and selected library dependencies. Aggregate results are recorded with their revision in `artifacts/native-verification.json`; the self-compilation record gives the relevant stage and binary hashes.

This manuscript builds from `paper/main.tex` and `paper/references.bib` with Tectonic. The public introduction at <https://etellis.github.io/U-web/> runs an interactive finite independence model written in U and compiled to WebAssembly. It checks that declared model's conditions; it does not issue general proof or physical-device evidence.

The provenance directory preserves the design documents, review refinements and 22 original examples. Implementation results bind to the tested revision. The repository's design history separately records the recovery of source material and the limits of the historical package.

12 Related work and research position

MLIR provides extensible typed operations, regions and progressive lowering across compiler and application domains [5]. U's shared graph representation is therefore not by itself a novelty claim. The research question concerns contracts made mandatory and inspectable across meaning, realization, source reconstruction and

evidence. A convincing evaluation must compare the cost and capability of that protocol with appropriate MLIR-based and domain-specific alternatives.

Algebraic effects and handlers already organize computation using operations, equations and interpretations into models [7]. U builds on the idea that interpretation must respect laws. Its proposed additional boundary is an integrated user-visible account of resource authority, physical realization, residuals and evidence. The paper does not claim that all seven theory families are one algebraic theory or that every combination admits a distributive law.

Structured cospans provide a principled account of open systems with interfaces and compositional network models [1]. They support the plausibility of common structural composition across scientific domains. U still owes concrete generators, operational rules, usability and adapter checks; the availability of a categorical construction does not automatically supply an executable language or the conditions for every mixed-domain composition.

Bidirectional transformation and lens work studies disciplined extraction and propagation of views [4]. U uses this perspective to separate byte reconstruction from semantic lifting, and to make source updates conditional on valid residual rules. The elementary lens proposition here is an application of those laws, not a new result about lenses.

Verified compilation work, exemplified by CompCert, treats semantic preservation as a theorem about concrete compiler passes and source/target semantics [6]. U’s conditional metatheorems and tests do not attain that level of whole-implementation correspondence. A future proof must connect the implemented graph, checker, evaluator and compiler passes to their formal models, with the bootstrap and external calls in the trusted base.

OpenQASM 3 distinguishes quantum computation, classical control and timing at multiple levels of circuit specificity [2]. It provides a concrete comparison target for the Q/C boundary. U must preserve a declared OpenQASM fragment’s semantics and execution assumptions before claiming native support; a Bell-state simulation or emitted circuit file does not establish that result for the full language or physical hardware.

The present originality claim is thus a research hypothesis about the integrated protocol and its execution discipline. Demonstrating useful compression without hiding domain semantics requires comparative artifacts, preservation evidence, usability work and realistic maintenance accounting.

13 Limitations and further work

The structural theorem depends on sound local generator rules and substitution. The resource theorem depends on unforgeable ownership and correct footprints. Adapter composition depends on a genuinely compatible intermediate observation. Identity invalidation depends on complete dependency capture and a finite collision assumption. Evidence non-fabrication depends on correct constructors and gates. These premises are obligations to implement and verify; writing them down does not prove them of every execution environment.

The current work does not establish a globally minimal semantic basis, automatic subsumption of arbitrary source languages, a complete dependent type theory, a production distributed runtime, hostile-package isolation, formal correspondence of all implementation code, or physical GPU/HDL/QPU execution. Full U1 compatibility, general dense spectral analysis and long-running memory reclamation also remain incomplete. Classical simulation, interchange emission, numerical calibration and physical realization are distinct support classes.

The development program connects foundational contracts and canonical syntax to executable values, resources and proofs; native CDC compatibility; theory conformance; faithful ingestion and law-aware transforms; production classical runtime; specialized execution; stronger formal correspondence; and continued self-hosting. The compiler and structural graph now execute in U, with reproducible stages and independent comparison. Formatting preserves comment and literal lexemes while canonicalizing the supported layout, with idempotence checked on the source corpus. Broader law-aware source transformations and general WebAssembly lowering remain further work. Independent implementations remain useful checks as the native

system grows.

Progress should be measured independently in compile time, runtime, memory, semantic coverage, representation density, proof/analysis coverage and human comprehension. Performance comparisons require matched semantics, numeric modes, hardware and workloads. A compact graph is not automatically faster execution, and the ability to describe a physical process does not establish a new physical law.

Authorship and disclosure

Edward Ellis is the author. AI systems assisted with design exploration, implementation, mathematical exposition, proof drafting and editorial review. Responsibility for the manuscript and its claims remains with the author. This research preprint is prepared for arXiv; it has not been submitted or peer reviewed.

A Theory accounting and refusal cases

Family	Native meaning to retain	Representative refusal or distinction
V	Natural data, closures, staged syntax, equality and admitted elimination	Partial computation does not enter total conversion; a quoted body does not execute merely because it is quoted.
M	Fresh allocation identity, layout, access, ownership and lifetime	Duplicate owner, escaped borrow, stale epoch or unadmitted raw access.
R	Variables, unification, relations, bags, search policy and fixed points	An occurs-check policy cannot silently become rational-tree unification; bags cannot become sets without a loss account.
P	Measures, likelihood weighting, conditioning and normalization	Zero or infinite normalization mass; sampler convergence cannot be inferred from a seed or a finite trace.
C	Event structure, mailbox policy, failures, fairness, clocks and registers	Trace order is not causal order; one finite schedule does not prove fairness or all-behavior equivalence.
D	Paths, equations, domains, guards, resets, derivatives and frames	A finite solver map does not inherit exact flow laws; changed itinerary or grazing blocks the corresponding derivative.
Q	Coherent joint state, channels, instruments, measurement and reset under the declared complex Hilbert-space profile	Classical circuit descriptions may be copied; an unknown physical state does not inherit copying or inspectable simulator storage.

Each family requires concrete operator accounting. The table records the architecture’s minimum distinctions; it does not report complete implementation of the listed semantics. A quantum density simulator should, for example, distinguish $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ from its dephased mixture. Both have the same computational-basis probabilities, but $\langle X \otimes X \rangle$ is one for the coherent Bell state and zero for the mixture. Matching only one measurement basis is an insufficient conformance test. This is a mathematical/classical simulation probe, not a report of physical nonclassical evidence.

B Formal claims and their exact boundaries

Claim	Indispensable premises	Excluded promotion
Structural preservation	Locally sound rules, matching boundaries, capture-free substitution and admitted iteration	Whole-runtime soundness or universal progress
Deterministic elaboration	Fixed manifest/version and deterministic resolution, binding, adapter choice and encoding	Semantic-equivalence decision or unique global normal form
Resource nonduplication	Fresh identities, consuming transfer, checked disjoint owners and unforgeable permissions	Complete alias analysis, distributed authority or physical simultaneity
Adapter composition	Same intermediate object/observations, compatible quantifiers, retained assumptions	Arbitrary combination of theories or unconditional equivalence
Approximation bound	Metric domains, uniform local error and valid Lipschitz constant	Global error from a local tolerance setting alone
Identity invalidation	Injective canonical encoding, complete dependency DAG and no encountered hash collision	Authentication, truth or arbitrary semantic equivalence
Earned artifacts	Fixed subject, correct gates, prerequisite checks and trusted constructors	Truth of untrusted evidence or rollback of prior effects
Residual reconstruction	Correct left inverse; valid lens and semantics factorization for edit claims	Native possession from byte retention
CDC return analysis	Actual local Jacobians, accepted itinerary, complete or justified relative recurrence	Floquet interpretation of an arbitrary nonreturning path

C Source and reproducibility record

The repository preserves the initial design and subsequent refinements to whole-surface elaboration, graph adapters, multidimensional admission, contract-sensitive meaning identity and conservative independence. These documents provide a versioned record of the design’s development.

The BiDi compatibility reference binds a specific revision of the source language, formal semantic spine, verification matrix, frameworks, U2 semantics, toolchain contracts and RFTC definitions. The architecture document links those files and identifies the corresponding U obligations. Comparisons against the independent runtime report their executed scope and numeric profile. Historical aggregate counts are distinct from those revision-specific results.

References

- [1] John C. Baez and Kenny Courser. Structured cospans. *Theory and Applications of Categories*, 35(48):1771–1822, 2020. <https://arxiv.org/abs/1911.04630>.
- [2] Andrew W. Cross, Ali Javadi-Abhari, Thomas Alexander, Niel de Beaudrap, Lev S. Bishop, Steven Heidel, Colm A. Ryan, Prasahnt Sivarajah, John Smolin, Jay M. Gambetta, and Blake R. Johnson. OpenQASM 3: A broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing*, 3(3), 2022. <https://arxiv.org/abs/2104.14722>.

- [3] Edward Ellis. BiDi universal operator system: Coherence-delta calculus, native runtime and U2 semantics. Software and repository documentation, 2026. Release 0.3.0; grammar 1; C ABI 1.5. Pinned commit 1307f2a7f32ab5beb64fe5cd0c0faf13ec9c8157. <https://github.com/ETEllis/BiDi>.
- [4] J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems*, 29(3), 2007. Article 17. Author manuscript: <https://www.cis.upenn.edu/~bcpierce/papers/lenses-full.pdf>.
- [5] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: A compiler infrastructure for the end of moore’s law. *arXiv preprint arXiv:2002.11054*, 2020. <https://arxiv.org/abs/2002.11054>.
- [6] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. <https://xavierleroy.org/publi/compcert-CACM.pdf>.
- [7] Gordon Plotkin and Matija Pretnar. Handlers of algebraic effects. In *Programming Languages and Systems (ESOP)*, volume 5502 of *Lecture Notes in Computer Science*, pages 80–94. Springer, 2009. Author manuscript: https://homepages.inf.ed.ac.uk/gdp/publications/Effect_Handlers.pdf.